Predicting the Lifetime of Flaky Tests on Chrome

Samaneh Malmir
Department of Computer Science
and Software Engineering
Concordia University
Montreal, QC, Canada
s_malmir@encs.concordia.ca

Peter C. Rigby
Department of Computer Science
and Software Engineering
Concordia University
Montreal, QC, Canada
peter.rigby@concordia.ca

Abstract

Flaky tests not only disrupt the testing process but can also introduce significant delays in software release cycles, leading to increased costs and potential missed market opportunities. The ability to predict the lifetime class to which the test belongs to can empower development teams to allocate resources efficiently, prioritize bug fixes, and streamline the testing pipeline. Despite the critical role that this estimation plays in software development, there is a noticeable gap in the existing research literature. Bridging this gap is essential for enhancing testing efficiency and optimizing resource allocation and project planning in software development.

In our investigation of the historical patterns of flaky tests in Chrome, we identified that 40% of flaky tests remain unresolved, while 38% are typically addressed within the initial 15 days of introduction. Subsequently, we developed a predictive model focused on identifying tests with quicker resolutions. Our model demonstrated a precision of 73% and a Matthews Correlation Coefficient (MCC) approaching 0.39 in forecasting the lifespan class of flaky tests.

Keywords

Flaky Tests, Lifetime Prediction, Software Testing, Chrome, Machine Learning, Lifetime Prediction

ACM Reference Format:
Samaneh Malmir and Peter C. Rigby. 2024. Predicting the Lifetime of Flaky Tests on Chrome. In *2024 International Flaky Tests Workshop 2024 (FTW '24), April 14, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3643656.3643899

1 Introduction

Continuous Integration (CI) is a vital software engineering process that facilitates collaboration by allowing frequent integration of code changes into a shared repository. By automating various facets of the development life cycle, CI enhances the speed and efficiency of software development [4]. An important aspect of this process is regression testing, where test suites are systematically executed for each commit, ensuring that new changes do not introduce faults or disrupt existing functionalities. However, the scalability and efficacy of traditional CI systems tend to diminish as codebases expand and change frequency intensifies. This challenge is especially notable in large, dynamic codebases like those at Google, which witness a high volume of code changes per minute and substantial file modifications each month [14]. Google addresses this by employing a unique CI system that uses dependency analysis to selectively execute tests affected by code changes. In this paper, we study the Chrome project that runs thousands of tests every day and publishes the result, analysis, bug-fixing process, and flaky test data publicly. The scale of this public data makes it an excellent source for investigating flaky test mitigation strategies and practices in a large-scale test environment.

In contrast to conventional testing approaches, Chrome employs a system with multiple "expected outcomes" for each test, documented in the `testExpectations` file. These anticipated outcomes include pass, fail, skip, crash, and timeout. Developers manually adjust the test expectation file as they modify their expectations for a specific test. Our dataset for this research comprises 78,450 tests with manually assigned expectations, among which 8,279 are labeled as "flaky." The dataset is derived from the historical changes made to this file. It's noteworthy that tests may have multiple outcomes, such as failure or crash, and developers update the file based on test results and their examination of failures.

1.1 Research Questions

RQ1: Descriptive Statistics: what is the frequency of each category of flaky test?

In order to prioritize tests for investigation, it is essential to understand the content of the `testExpectations` file, the distribution of the test categories, and the time it takes for a flaky test on Chrome to be resolved. Addressing this question is key to devising an effective framework for approaching the problem.

Results summary. We find that 40% of the flaky tests are never fixed. Of the flaky tests that are fixed or removed, 38% are in less than 15 days.

RQ2: Flakiness Survival: How has the number of flaky tests changed over time?

Most previous research has primarily focused on the root causing of flaky tests *e.g.,* [11, 14, 21] and flakiness detection [12] [9, 20]. In contrast, our goal is to comprehend the duration for which tests remain flaky.

We analyze the entire history of changes in the `testExpectations` file, over a period of 3 years. We are able to determine how long a test remains flaky by its insertion, removal, or change in test expectation.

Results summary. Our investigation reveals that except for two short-lived spikes in a period of 3 years, the total number of flaky

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FTW '24, April 14, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0558-8/24/04…$15.00
https://doi.org/10.1145/3643656.3643899

tests is roughly constant.

RQ3: Flaky lifetime prediction: How accurately can we predict the lifetime of a flaky test based on the issue description?

Chrome developers manually investigate and fix flaky tests. We want to automatically suggest tests that are quick to fix based on historically mined data.

We use a Random Forest model, with the outcome being the class that the lifetime of a flaky test belongs to between 'quick to fix' and 'slow to fix/not worthy of investigation'.

Results summary. Test description is the most important feature in predicting the lifetime of flaky tests. Our model can predict the lifetime class of flaky tests with a precision of 73% using the description of tests.

The rest of the paper is organized as follows: Section 2 provides an introduction to the dataset utilized and the features incorporated in the study. Sections three through five are dedicated to an in-depth exploration of the research questions, the applied research methodology, and the ensuing results. Section 6 addresses potential threats to the validity of the study. In Section 7, we engage in a discourse on the literature, drawing comparisons with our research. The final section encapsulates the conclusions drawn from the study's findings.

2 Background and Data

The Chrome browser is a Google-lead, cross-platform, open-source web browser. As of April 2022, StatCounter estimates that Chrome has a 64% worldwide browser market share.[1] We selected the Chrome project for our research because the data is publicly available and the number of flaky tests is much higher than projects being used in recent academic works on flaky tests [5, 19]. To elaborate on the scale, we note that in a period of one month, January 2021, the Linux builder for the Chrome project contains 27M test runs with 66k failures and 21k flaky tests. In contrast, [19] found only 29 flaky failures by rerunning 252 failing CI jobs.

2.1 Determining Test Categories and Build Status

Figure 1 outlines the decision-making procedure employed by LuCI (Chromium CI platform) to determine the result of a specific test within a build. A test is considered successful if it passes without errors in a single execution. In case of failure, LuCI initiates automatic retesting, repeating the test up to five times. If all reruns fail, the test is categorized as "unexpected", leading to a build failure. Furthermore, if a test succeeds despite having experienced one or more failed executions within the same build, it is classified as "flaky." However, this outcome does not hinder the overall successful progression of the build [6].

2.2 Allowable Test Outcomes

We note that in prior work, the number of flaky tests per project was small. For example, Moritz et al. [5], studied Mozilla in 2019, which has 100 to 150 new flaky tests every week. In comparison, on Chrome, there are approximately 700 new flaky tests every day. This difference can be the result of the extensive codebase and a large number of contributors to the Chromium project. Also, Chrome's rapid release cycle and continuous integration practices might introduce changes more frequently, increasing the chances of test flakiness. The scale of flaky tests on Chrome forces developers to find new ways to reduce the impact of flakiness on the CI environment. The *testExpectations* file is their solution. It records the expected outcomes that developers have manually allowed for each test. Five possible expected outcomes can be manually assigned to a test:

Pass: a test passes when its output matches the pre-defined expected results. By default, tests are expected to pass. If a test is not named in the *testExpectations* file, then only Pass is an expected outcome.

Failure: a test fails when the outcome of the test does not match the defined expected result that resides next to the test file in the test directory.

Skip: Tests marked as Skip will not be run by default, generally because they are not appropriate given the environment. For example, Windows-specific tests are skipped on Linux and Mac builds.

Timeout: Tests that take longer than a certain amount of time to complete are aborted and marked as "timed out". For example, some tests can trigger significant memory allocation in the GPU process, leading to intermittent Out-Of-Memory (OOM) errors and GPU process crashes which yield a TIMEOUT outcome. Tests that commonly time out can be labeled and added to the `testExpectations` file to reduce re-runs.

Crash: Tests that cause the test shell or browser to crash.

As an example of test expectations, below we show the line that allows the expected outcome of the test on Mac11-arm64 OS to be either a Failure or a Pass. In effect, the developers have decided to ignore this flaky test. However, the test is still run once so that developers can manually examine the output. The test will not block the build.

```
+crbug.com/1302856 [ Mac11-arm64 ]
    virtual/exotic-color-space/images
  /rgb-png-with-cmyk-color-profile.html [ Failure Pass ]
```

as another example, the following line has been added to the file to prevent crashing until it gets fixed. For this change, developers added the following description:

"This CL introduces a change to the maxframes.https.html test that sets the 'src' attribute on a fenced frame element that is created in a page that has surpassed the subframe limit. The renderer crashes in this case, so this CL adds TestExpectations to cater to that until the problem is fixed. " [2]

```
+crbug.com/1123606 virtual/fenced-frame-mparch
/wpt_internal/fenced_frame/maxframes.https.html [ Crash ]
```

By labeling these tests in the file, the tests will only be run once and will always be categorized as "Expected".

We mine the entire history of tests added and removed to the *testExpectations* file from November 2018 to March 2022 to understand how flakiness has affected Chrome over time. During this

[1] https://gs.statcounter.com/

[2] https://chromium-review.googlesource.com/c/chromium/src/+/3501941

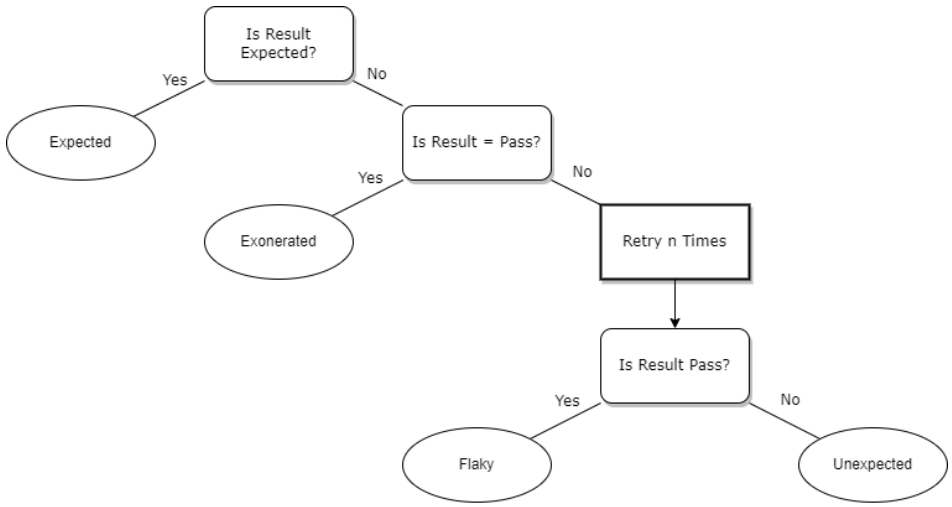


Figure 1: A decision tree showing how test label is determined in Chromium. If the result is an unexpected pass it is labeled as exonerated. In case it fails, the builder reruns it n times and if one of the results is a pass, it is considered flaky. Otherwise, it's an unexpected failure. [6]

time 78,452 changes were made to this file which included adding new tests, removing old tests, and changing the expectation for a test.

Even though the flaky tests do not stop the build, the tests are examined in the Chromium Gerrit, Chromium code review system, and a bug report is created for the flaky tests.

3 RQ1: Descriptive Statistics

The goal of this research question is to understand the content of the *testExpectations* file and investigate the distribution of expected outcomes.

3.1 What is the frequency of each category of flaky test?

Figure 2 demonstrates the frequency of different categories of expected outcomes in the *testExpectations* file. As we can see, "Failure" is the most occurring expected outcome, with 16K tests accounting for 40% of tests in the file. Failures block the build and must be fixed before integrating the change.

The Flaky category is an aggregated group of different test outcomes that include "Pass" and some kind of failure, including failure pass, pass timeout, failure pass timeout, and crash pass. In Figure 2 we can see that 30% of the added tests to the *testExpectations* file are flaky.

The next common category is 'Timeout". 'Timeout" is the expected outcome for the tests that take a long time to finish, So they are aborted by the test runner. Almost 8% of the tests added to the *testExpectations* file expected to timeout.

When there is a test that developers want to be disabled temporarily, they add it to this file with 'Skip" outcome. According to Figure 2, 3.3k tests added to the *testExpectations* file with 'Skip" as expected outcome.

3.2 How many flaky tests lead to a bug report?

Table 1 categorizes test outcomes based on the number of bug reports associated with each test. The data indicates that 80% of flaky tests are linked to a bug ID, which is higher than other types of failed tests. This suggests that the nondeterministic nature of flaky tests makes them more challenging to fix compared to regular failures, resulting in a higher number of bug reports to track the tests over time.

Platform-specific tests are those that the expected outcome happens only on a specific platform, for example, the test is expected to fail only on Windows. By looking at this table, we see that there is a link between the percentage of cross-platform tests and the percentage of bug IDs assigned. This means that when a test shows an unexpected behavior only on a specific platform, it is more likely to be ignored by developers.

> `testExpectations` file is the main test failure suppression method and is used to temporarily accept the expected outcome as the test result until it gets fixed. "Failure" and "Flaky" are the two most common outcomes in the file, accounting for a combined total of 70% of all tests.

4 RQ2. Flaky Survival

How has the number of flaky tests changed over time?

This research question aims to analyze the patterns and trends in the number of flaky tests over time. Our objective is to understand how the count of currently monitored flaky tests fluctuates with the introduction of new instances and the resolution of existing ones by developers. Addressing this question provides insights into the scale of the flaky test issue in Chrome. For instance, an upward trend suggests a persistent challenge, indicating that the resources,

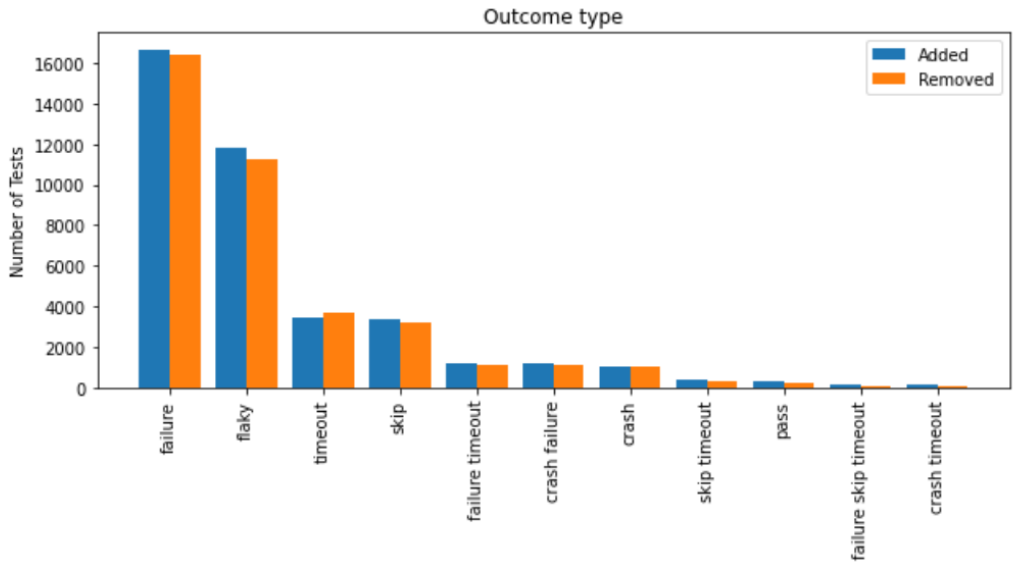


Figure 2: Frequency of different outcomes in test expectation file. Failed tests and flaky tests are the most occurring categories with 40% (16K tests) and 30% (12K tests) of the tests in the file accordingly.

Table 1: Chromium Dataset Summary: Flaky is the second most occurring outcome, and 81% of flaky tests have a bug ID assigned to them to track the test over time, whereas this number for failed tests is 68%.

Outcome	Count	Bug	BugPercentage	CrossPlatformPercentage
Failure	19,153	13,151	68%	68%
Flaky	12,693	10,306	81%	54%
Skip	4,171	3,881	93%	60%
Timeout	3,597	1,358	37%	25%
Crash	1,089	498	45%	38%

including developers' time, allocated to investigating flaky tests may be insufficient.

The "lifetime" of a flaky test is defined as the period from its inclusion in the `testExpectations` file until the time at which developers investigate and resolve it, leading to its removal from the file. To answer this research question, we order tests based on the date they are added to the expectations file. The initial count of flaky tests is determined by the number of tests presented on the earliest day available in the changelog. Then for each subsequent day, we update this count by adding the number of new flaky tests and subtracting the number of fixed flaky tests on that particular day. Eventually, we generate the cumulative chart for the number of existing flaky tests over a period of 3 years.

4.1 Cumulative Chart of Flaky Tests Count

Figure 3 illustrates the cumulative count of flaky tests over 3 years. The graph reveals noticeable spikes and dips on specific dates, indicating substantial fluctuations. These variations align with builds that introduce a considerable number of new flaky tests, attributed to the incorporation of additional, or removing a group of flaky tests due to enabling or disabling a feature on the platform under study. For example, on January 3, 2020, 583 flaky tests were added to the file because of enabling visually-refreshed form controls on Windows/ChromeOS/Linux. The developers commented, "Because many layout tests use form controls, including tests that are not directly testing form controls, this CL [pull-request] requires around 1200 tests to be rebaselined." Then they create different bug issues to track the process of cleaning these newly added tests and remove these new lines from TestExpectations.

`testExpectations` file is the main test failure suppression file and is used for temporarily marking tests as flaky until they get fixed. Our investigation reveals a consistent trend: the quantity of existing flaky tests remains relatively stable. This stability implies that the number of resolved flaky tests is nearly equivalent to the number of newly identified flaky tests introduced over time [15].

4.2 Survival Chart of Flaky Tests Lifetime

Figure 4 represents the survival probability of flaky tests over time. The survival chart represents the proportion of subjects that have not encountered the event (in our case, the tests that have not been fixed) up to a certain point in time. As time progresses, the survival probability decreases, indicating the proportion of tests that have not survived (been fixed) by each time point.

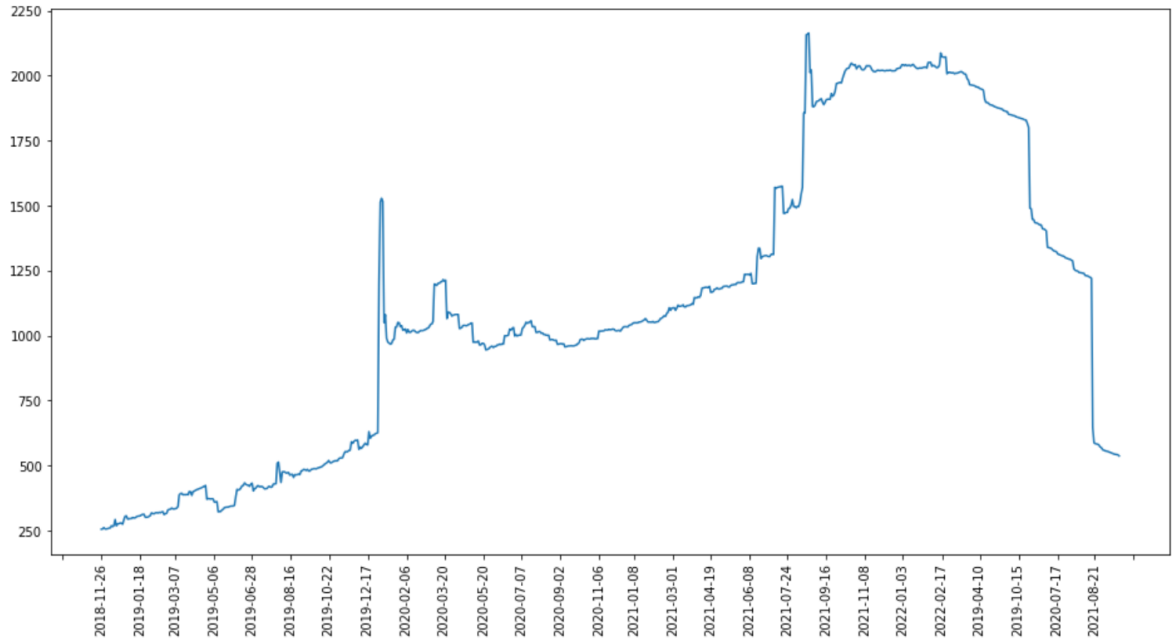


Figure 3: Accumulated Number of Flaky Tests Over Time. The steep rise in early January 2020 is due to the visually-refreshed form controls feature activation, which led to the addition of 583 tests to the file until their baseline was updated.

On day one, we have a total of 8,279 tests. In the early days (between 10–15 days), there is a relatively steep decline in survival probability. This suggests a higher rate of tests getting fixed early on. By looking more closely at the numbers, we observe that 33% of the tests get fixed before 10 days and 38% before 15 days. After the initial decline, there is a period where the survival probability plateaus or decreases at a slower rate. This suggests the tests, that do not get fixed by a certain point in time, have a higher probability of being left unresolved or getting fixed very late. We can consider that these tests do not have a high priority for developers. There is a second, more gradual decline in survival probability starting around day 200. This indicates that over a more extended period, additional tests are getting resolved. Toward the end, the number of remaining tests doesn't reach zero, constituting 40% of all tests, suggesting some tests remain unfixed. The median survival time is the point at which 50% of the tests have been fixed. We can see that the median survival appears to be between 100 and 200 days, more accurately at day 122. The overall trend shows a decreasing trend in the number of available tests for debugging, with a higher rate in the first 15 days and a more stable trend after 200 days.

Analyzing the evolution of changes in `testExpectations` file over time reveals that except for two short-lived spikes in 3 years, the total number of flaky tests is roughly constant. By studying the survival time of flaky tests, we find that 40% of flaky tests are never fixed. Of the flaky tests that are fixed or removed, 38% are in less than 15 days.

5 RQ3. Flaky Lifetime Prediction

5.1 Model definition and outcome measures

Our objective is to develop a model to predict the lifetime class of a flaky test. We view the lifetime of a flaky test as a valuable metric for identifying tests that can be resolved quickly, thus helping developers optimize their time allocation.

We describe our model which uses a column transformer of textual data to vectors, a feature selector that selects the best K features by applying a Random Forest Classifier. To represent the descriptions, we employ CountVectorizer for the conversion of texts into a matrix that captures token counts. This method, commonly referred to as the "bag-of-words" approach, has been employed in prior vocabulary-based methodologies that identify flaky tests [7, 16, 20]. Initially, these vectors include features equivalent to the number of words in the test descriptions. As the resulting dictionary may become sizable, it becomes essential to employ feature selection techniques. These techniques aim to reduce the dictionary's size, eliminate irrelevant features (thus minimizing noise in the data), and choose the most informative ones. Therefore, feature selection plays a crucial role in decreasing model training time.

We employ the SelectKBest method [13], which preserves the top k features with the highest scores, determined through a univariate statistical test using χ^2. Fine-tuning the hyperparameters of our machine-learning pipeline involves adjusting parameters such as the number of trees in the forest, the sampling strategy for SMOTE (Synthetic Minority Oversampling TEchnique), and the number of features to retain. After achieving the optimal settings, we proceed to retrain a model on the entire training set and subsequently assess its performance using the holdout dataset.

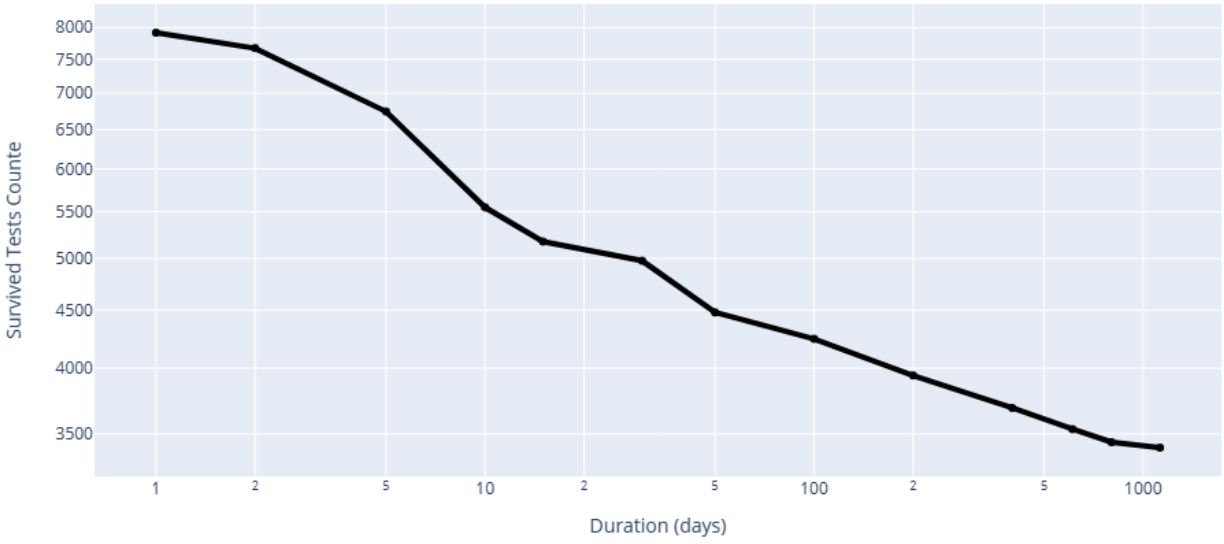


Figure 4: Log Scale Survival Chart of Lifetime of Flaky Tests: Approximately 40% of tests get fixed within 15 days. nearly 40% of the tests survive by the end of the period.

To develop our predictive model, we use the entire set of flaky tests as our dataset. We examine the distribution shown in Section 4.2, and note that 40% of flaky tests never get fixed. Of the remaining 60%, more than half are fixed within 10 to 15 days. We categorize tests into two classes based on the duration it takes to resolve the flaky test: tests that will be fixed "quickly" and those that will take a long time to fix or will never be fixed, i.e., fixed "slowly". Our primary goal is to develop a model that distinguishes the "quick" class from the "slow".

We defined "quick10" to be tests that are fixed in $\leq$ 10, and "quick15" to be tests that are fixed in $\leq$ 15 days. We developed two models: one to predict tests fixed within 10 days (referred to as "quick10") and another for tests resolved within 15 days (referred to as "quick15"). In both cases, the categories the model predicts are that a flaky test will be fixed quickly or slowly. The model demonstrating superior performance among the two settings was selected as our final model.

Outcome measures. We use standard evaluation metrics, precision, recall, F1 score, and Matthews correlation coefficient (MCC), to evaluate the performance of our model and compare different classifiers. These metrics have been used to evaluate the performance of classifiers, including binary classification of flaky tests [20] [7] [2]. Since our dataset is unbalanced, we use a weighted metric (MCC) for our evaluation.

Training and test data. Our data division strategy involves partitioning it into two segments: the initial 80% of tests constitute our training set, while the remaining 20% form the holdout set. This approach ensures that we maintain the temporal evolution of the text expectation file over time and mitigates any potential data leakage issues that might arise from random selection. Emphasizing this temporal aspect is crucial, as our observations indicate that failing to account for this condition and training a model on a shuffled dataset would lead to a significant overestimation of performance.

5.2 Results

Table 2 reports the obtained performance. We achieved a promising precision rate of 73.6% when categorizing tests into two distinct classes: "quick to fix" and "slow to fix", with "quick to fix" being the tests fixed within 15 days. Within the scope of our research, precision holds paramount significance, as it dictates the proportion of tests labeled for investigation that truly warrant priority attention. A precision rate of 73% signifies that when we recommend a test for investigation, it proves to be genuinely "quick to fix" approximately 73% of the time, ensuring that our suggestions to developers are mostly accurate and avoid unnecessary investigations.

To optimize our model, we engaged in parameter tuning, which we will discuss in the following. Table 3 presents a comprehensive overview of our predictive model's performance, highlighting the systematic adjustment and evaluation of various parameters and features. In the following, we delve into a discussion of these outcomes to better understand the model's behavior:

The base model achieved a decent level of precision (67%) and recall (62%), indicating that it could effectively identify the target

Table 2: Result of the model, predicting tests fixed in $\leq$ 15 days (quick15) and $\leq$ 10 day (quick10)

Model	Precision	Recall	MCC	F1-Score
quick15	73.6%	57.4%	0.39	64.5%
quick10	62.7%	40%	0.28	49%

variable. The MCC score of 0.34 suggests a moderate level of agreement between predictions and actual outcomes. The F1 score of 0.64 demonstrates a balanced performance in terms of both precision and recall.

However, the introduction of the Author and Expectation features adversely affected the model's performance, resulting in a decline across all four metrics. Although subsequent Feature Selection implementation showed improvements, it only brought the performance up to the base model's levels, suggesting a limited contribution of these features to overall model performance.

By expanding the maximum number of features to 400, a marginal improvement in precision was noted. Additionally, aligning the number of estimators with the maximum features resulted in a notable precision of 73.6%, indicating an enhanced ability to identify positive cases. These findings imply that a more extensive feature set led to improved model performance, which is reasonable considering the textual nature of our data.

Finally, the application of oversampling resulted in substantial improvements, with the MCC score reaching 0.397 and the F1 score demonstrating a notable increase to 64.5%.

In summary, the experimental variations in parameters and features provided valuable insights into model performance. Feature selection appeared to positively impact prediction accuracy, while an increased feature count contributed to higher precision and recall. The combination of oversampling techniques and parameter tuning also led to notable improvements.

> Using only the description of a test, we can predict the lifetime of flaky tests with 73% precision and MCC close of 0.39.

6 Threats to Validity

Generalizability. When predicting how long it takes to fix flaky tests based on past changes, it is important to consider if the findings apply to different projects and how human factors may influence resolution times. Project variations are significant, and what works for one may not work for others. Researchers need to carefully explore if models from one dataset can be used for different projects, and be transparent about any limitations in generalizing the results.

Construct Validity. It is crucial to recognize how human factors can affect the time it takes to fix flaky tests. Changes in development teams and project management practices can impact how quickly issues, including flaky tests, are addressed. Considering and understanding these human-related factors is essential for predicting fix times accurately. Researchers should be vigilant in their analysis to understand the effects of these factors, which significantly impact the reliability of predictive models.

Internal Validity. To explore the impact of random dataset splitting, we examined two configurations: one involving a temporal dataset split, where the initial 80% of the data served as the training set, and the remaining portion as the test set, and the other utilizing a 5-fold cross-validation. In the first configuration, we trained our model on 6,600 tests, then, we evaluated it on 1,650 tests. In the second setting, we partitioned the dataset into five equally sized folds. The model is trained on four of the folds and tested on the remaining fifth fold. This process is repeated five times, each time using a different fold as the test set. We achieved 0.65% Precision, 0.25 MCC, 0.48 Recall, and an Accuracy of 0.62% using 5-fold cross-validation, on our best model. This suggests that considering the temporal splitting preserves the nature of data, providing a more realistic evaluation of the model's performance.

7 Literature and Discussion

Flaky Tests Prediction. Pinto et al. [20] employed a bag of word representation for each test to develop a model capable of identifying flaky tests using the vocabulary extracted from the test code. The authors report a high F-measure of 0.95 in the prediction of flaky tests. In their study, Camara et al. [3] conduct a replication to evaluate the effectiveness of code identifiers in predicting test flakiness across various datasets. The findings of their research highlight that the vocabulary of flaky tests is significantly influenced by the context of a project. Consequently, the defined vocabulary lacks the necessary level of generalization to reliably predict flaky tests in other contexts. Flakify, proposed by Fatima et al. [7] is a language model-based predictor for flaky test cases, developed by fine-tuning CodeBert, a pre-trained language model, using solely the source code of flaky tests. The research outcomes demonstrate that Flakify achieves promising F1 scores on both the FlakeFlagger and IDoFT datasets, resulting in significant reductions in the costs associated with debugging test cases and production code. Specifically, Flakify reduces the debugging cost by 25% and 64% for the test cases and production code of FlakeFlagger, respectively. While these studies inspired our exploration of vocabulary-based methods, none of them specifically addressed the prioritization of flaky tests for investigation and resolution.

The work of Pinto et al. [20] was further replicated by Haben et al. [8] to explore the impact of different features extracted from the Code Under Test and a time-sensitive train/test split. The authors of the replication study found that the information in the Code Under Test had a limited influence on the performance of the vocabulary-based models. Nevertheless, they concluded that vocabulary-based models can still be employed to predict test flakiness in other programming languages.

Flaky Tests Ranking. In the study by Aman et al. [1], a variety of Natural Language Processing (NLP) techniques were employed to rank the test cases. Lachmann et al. [10] compiled a dictionary of commonly occurring terms within the descriptions of all test cases, along with execution history, to prioritize the test cases using a ranked Support Vector Machine (SVM). In contrast to prioritizing tests for execution, our approach focuses on ranking tests for investigation. We believe that our method can enhance the test execution

Table 3: Result of using different configurations and features. The Base Model has 200 estimators and a maximum feature count of 200 and has quick15 as the target class.

Model	Precision	Recall	MCC	F1-Score
Base Model	67%	62%	0.34	0.64
Base Model with Expectation and Author features	63%	58%	0.27	0.61
Base Model with Expectation and Author features + Feature Selection	66%	62%	0.35	0.64
Base Model + Oversampling	67%	63%	0.34	0.64
Base Model + Oversampling + Feature Selection	67%	60%	0.34	0.64
Base Model (Max Feature = 400)	68.8%	59.3%	0.35	63.7%
A: Base Model (Max Feature = 400, N estimators = 400)	73.6%	57.4%	0.35	63.7%
Final Model: Model A + oversampling rate = 0.8	73.6%	57.4%	0.39	64.5%

ranking efficiency. In [6], Fallahzadeh et al. apply test prioritization methods on test runs to rank tests for execution whereas in our approach we prioritize flaky tests for investigation.

Overall, despite the potential of NLP techniques, the current use of NLP in a Test Selection and Prioritization context is very limited [17].

Among the numerous methods for predicting flakiness, vocabulary-based approaches [20] - [8] [22] [16] stand out as the most widely adopted [18]. These methods rely on machine learning models to forecast test flakiness by analyzing the occurrences of source code tokens within candidate tests. Intriguingly, prior research has identified these approaches as remarkably accurate, with the current state-of-the-art achieving accuracy levels surpassing 95% [20], [8], [3], [7], [2].

Simultaneously, vocabulary-based approaches possess characteristics of being both static and text-based. Consequently, they exhibit traits of portability, meaning they are tailored to a specific programming language, and interpretability, enabling users to discern the factors contributing to flakiness based on the keywords influencing the model's decisions. This combination of precision, portability, and interpretability renders vocabulary-based approaches highly appealing, as they offer flexibility and ease of practical application.

8 Conclusion

While software testing has long acknowledged flakiness as a prominent issue, research on this subject has rarely focused on classifying flaky tests for investigation in a large-scale industrial CI setting. In this research, we collected the history of more than 8K flaky tests on the Chrome project to be utilized for our lifetime classification model. We observe that 40% of the flaky tests remain unresolved. Among the flaky tests that undergo resolution or removal, 38% are addressed within a timeframe of fewer than 15 days. Identifying the appropriate tests to address is crucial, given that we aim to avoid allocating developers' time to the 40% of tests that may prove unsolvable. Our preference lies in pinpointing tests that are both more manageable and hold greater importance for timely resolution.

To effectively prioritize tests for investigation, it is crucial to analyze the distribution of the time it takes for a flaky test in Chrome to be resolved. This exploration is fundamental for establishing a robust framework to address the underlying issues. By analyzing the distribution of fixing time of flaky tests we observe that except for two brief spikes observed over a 3-year period, our analysis indicates that the overall count of flaky tests remains relatively stable. This trend demonstrates that while developers address flaky tests, new ones continue to arise, maintaining a relatively consistent count of existing flaky tests. This shows the necessity for an efficient test prioritization method to direct developers' efforts towards reducing this trend.

We consider the lifetime of a test as a measure of the effort needed to address the test and its priority for being investigated. By solely analyzing the test description, our model can accurately predict the lifespan of flaky tests, achieving a precision of 73% and an MCC of 0.39. Analyzing test descriptions alone yields a high-precision method for predicting flaky test lifespans, suggesting a practical approach to test management.

References

[1] Hirohisa Aman, Sousuke Amasaki, Tomoyuki Yokogawa, and Minoru Kawahara. 2020. A comparative study of vectorization-based static test case prioritization methods. In *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 80–88.

[2] Bruno Camara, Marco Silva, Andre Endo, and Silvia Vergilio. 2021. On the use of test smells for prediction of flaky tests. In *Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing*. 46–54.

[3] Bruno Henrique Pachulski Camara, Marco Aurélio Graciotto Silva, Andre T Endo, and Silvia Regina Vergilio. 2021. What is the vocabulary of flaky tests? an extended replication. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 444–454.

[4] Paul M Duvall, Steve Matyas, and Andrew Glover. 2007. *Continuous integration: improving software quality and reducing risk*. Pearson Education.

[5] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests: The developer's perspective. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 830–840.

[6] Emad Fallahzadeh and Peter C Rigby. 2022. The impact of flaky tests on historical test prioritization on chrome. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 273–282.

[7] Sakina Fatima, Taher A Ghaleb, and Lionel Briand. 2022. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Transactions on Software Engineering* (2022).

[8] Guillaume Haben, Sarra Habchi, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2021. A replication study on the usability of code vocabulary in predicting flaky tests. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 219–229.

[9] Tariq M King, Dionny Santiago, Justin Phillips, and Peter J Clarke. 2018. Towards a bayesian network model for predicting flaky automated tests. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 100–107.

[10] Remo Lachmann, Sandro Schulze, Manuel Nieke, Christoph Seidl, and Ina Schaefer. 2016. System-level test case prioritization using machine learning. In *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 361–368.

[11] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root causing flaky tests in a large-scale industrial setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–111.

[12] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A framework for detecting and partially classifying flaky tests. In *2019 12th ieee conference on software testing, validation and verification (icst)*. IEEE, 312–322.

[13] Scikit learn developers. 2022. *Sklearn.feature_selection.selectkbest — scikit-learn 1.1.2 documentation.* https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html (Accessed on 08/11/2022), Aug. 2022 (cit. on p. 131).

[14] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 643–653.

[15] John Micco. 2016. *Flaky Tests at Google and How We Mitigate Them.* https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[16] Doriane Olewicki, Mathieu Nayrolles, and Bram Adams. 2022. Towards language-independent brown build detection. In *Proceedings of the 44th International Conference on Software Engineering*. 2177–2188.

[17] Rongqi Pan, Mojtaba Bagherzadeh, Taher A Ghaleb, and Lionel Briand. 2022. Test case selection and prioritization using machine learning: a systematic literature review. *Empirical Software Engineering* 27, 2 (2022), 29.

[18] Owain Parry, Gregory M Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–74.

[19] Qianyang Peng, August Shi, and Lingming Zhang. 2020. Empirically revisiting and enhancing IR-based test-case prioritization. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 324–336.

[20] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the vocabulary of flaky tests?. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 492–502.

[21] Alan Romano, Zihe Song, Sampath Grandhi, Wei Yang, and Weihang Wang. 2021. An empirical analysis of UI-based flaky tests. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1585–1597.

[22] Roberto Verdecchia, Emilio Cruciani, Breno Miranda, and Antonia Bertolino. 2021. Know you neighbor: Fast static prediction of test flakiness. *IEEE Access* 9, 76119–76134.